

Numerical Questions & Solutions

1. **Question: Return Calculation:** An agent receives a sequence of rewards: $R_1 = 2, R_2 = -1, R_3 = 5$. The episode ends after R_3 . Calculate the return G_0 assuming $\gamma = 1$.

Answer: $G_0 = R_1 + R_2 + R_3 = 2 + (-1) + 5 = 6$.

2. **Question: Discounted Return:** For the same sequence ($R_1 = 2, R_2 = -1, R_3 = 5$), calculate G_0 with a discount factor $\gamma = 0.5$.

Answer: $G_0 = R_1 + \gamma R_2 + \gamma^2 R_3 = 2 + 0.5(-1) + 0.25(5) = 2 - 0.5 + 1.25 = 2.75$.

3. **Question: Infinite Horizon Return:** A continuing task yields a constant reward $R_t = 10$ for all t . If $\gamma = 0.9$, what is the total return G_t ?

Answer: $G_t = R/(1 - \gamma) = 10/(1 - 0.9) = 10/0.1 = 100$.

Derivation (Sutton & Barto, Ch 3, Section 3.3, Eq. 3.10):

The return for a continuing task is $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$

When $R_t = R$ (constant) for all t :

$$G_t = R + \gamma R + \gamma^2 R + \dots = R(1 + \gamma + \gamma^2 + \dots) = R \sum_{k=0}^{\infty} \gamma^k$$

This is a geometric series that converges to $\frac{1}{1-\gamma}$ when $|\gamma| < 1$.

Therefore: $G_t = \frac{R}{1-\gamma}$

4. **Question: Expected Reward:** Given the dynamics $p(s_1, 5 \mid s, a) = 0.3$ and $p(s_2, -2 \mid s, a) = 0.7$, calculate the expected reward $r(s, a)$.

Answer: $r(s, a) = 0.3(5) + 0.7(-2) = 1.5 - 1.4 = 0.1$.

5. **Question: State-Transition Probability:** Using the dynamics from question 4, what is the state-transition probability $p(s_1 \mid s, a)$?

Answer: $p(s_1 \mid s, a) = \sum_r p(s_1, r \mid s, a) = 0.3$.

6. **Question: Bellman Equation (State Value):** In a gridworld, state S has four possible successor states with values $V(S_{up}) = 10, V(S_{down}) = 2, V(S_{left}) = 0, V(S_{right}) = 5$. If the policy π is uniform random and $\gamma = 0.9$ (with $R = 0$ for all moves), calculate $V(S)$.

Answer: $V(S) = \sum_a \pi(a \mid s)[R + \gamma V(s')] = 0.25[0 + 0.9(10 + 2 + 0 + 5)] = 0.25[0.9(17)] = 0.25[15.3] = 3.825$.

7. **Question: Bellman Equation (Action Value):** Given $p(s', 10 \mid s, a) = 1.0$ and $V(s') = 20$, calculate $q_\pi(s, a)$ with $\gamma = 0.9$.

Answer: $q_\pi(s, a) = R + \gamma V(s') = 10 + 0.9(20) = 10 + 18 = 28$.

8. **Question: Policy Evaluation Update:** A state s has only one action a . $p(s, 1 \mid s, a) = 0.2$ and $p(s', 10 \mid s, a) = 0.8$. If current estimates are $V(s) = 0$ and $V(s') = 5$, what is the new estimate $V(s)$ after one update with $\gamma = 1$?

Answer: $V(s) = 0.2[1 + 1(0)] + 0.8[10 + 1(5)] = 0.2[1] + 0.8[15] = 0.2 + 12 = 12.2$.

9. **Question: Value Iteration Update:** State s has two actions, A and B . Action A : Leads to s_1 (value 10) with reward 0. Action B : Leads to s_2 (value 5) with reward 6. Assume $\gamma = 0.9$. Calculate the new value $V(s)$ using the Value Iteration update.

Answer: $V(s) = \max\{0 + 0.9(10), 6 + 0.9(5)\} = \max\{9, 6 + 4.5\} = \max\{9, 10.5\} = 10.5$.

10. **Question: Greedy Action Selection:** Given $q(s, a_1) = 4.5, q(s, a_2) = 5.2, q(s, a_3) = 5.1$, which action is chosen by a greedy policy $\pi'(s)$?

Answer: a_2 (it has the highest Q-value: 5.2).

11. **Question: Gridworld Boundary Reward:** Consider a 3×3 gridworld with a uniform random policy ($\pi = 1/4$ for each action: N, S, E, W). The rules are:

- Moving off the grid: agent stays in the same cell and receives $R = -1$.
- Moving to any valid non-terminal cell: $R = 0$.
- Moving into the goal state at $(2, 2)$: $R = +10$, episode ends. $V(\text{terminal}) = 0$.
- $\gamma = 0.9$.

State values $V(s)$ and action values $Q(s, a)$ are given (see question sheet).

(a) Draw the grid showing which actions hit boundaries for state $(0, 2)$.

(b) Compute $Q((0, 2), \text{North})$ using the formula $Q(s, a) = R + \gamma V(s')$.

(c) Compute all four $Q((0, 2), a)$ values, then verify $V(0, 2) = \frac{1}{4} \sum_a Q((0, 2), a) = 0.56$.

Answer (Sutton & Barto, Ch 3, Section 3.5 - Bellman Equation for v_π):

(a) Grid diagram for state $(0, 2)$:

	Col 0	Col 1	Col 2	
				↑ North hits boundary ($R=-1$, stay)
	+-----+-----+-----+			
Row 0	-0.86	0.06	0.56	East hits boundary ($R=-1$, stay)
	+-----+-----+-----+			
Row 1	0.06	1.62	3.54	← South goes here ($R=0$)
	+-----+-----+-----+			
Row 2	0.56	3.54	0.00	
	+-----+-----+-----+			
		↑		West goes to $(0,1)$ with $R=0$

State $(0, 2)$ is in the **top-right corner**. Two actions hit boundaries:

- **North:** hits top boundary $\rightarrow$ stays at $(0, 2)$, $R = -1$
- **East:** hits right boundary $\rightarrow$ stays at $(0, 2)$, $R = -1$
- **South:** valid $\rightarrow$ goes to $(1, 2)$, $R = 0$
- **West:** valid $\rightarrow$ goes to $(0, 1)$, $R = 0$

(b) Computing $Q((0, 2), \text{North})$:

The action-value function is defined as: $Q(s, a) = R + \gamma V(s')$

For north from $(0, 2)$: agent stays at $(0, 2)$, $R = -1$, $V(s') = V(0, 2) = 0.56$

$Q((0, 2), \text{North}) = -1 + 0.9 \times 0.56 = -1 + 0.504 = -0.496 \approx -0.49 \checkmark$ (matches Q-table)

(c) All four Q-values and verification of V:

Action	Next state	R	$V(s')$	$Q(s, a) = R + \gamma V(s')$
North	$(0, 2) - \text{boundary}$	-1	0.56	$-1 + 0.9(0.56) = -0.49$
South	$(1, 2) - \text{valid}$	0	3.54	$0 + 0.9(3.54) = 3.18$
East	$(0, 2) - \text{boundary}$	-1	0.56	$-1 + 0.9(0.56) = -0.49$
West	$(0, 1) - \text{valid}$	0	0.06	$0 + 0.9(0.06) = 0.06$

These match the Q-table given in the question: $Q((0, 2), \cdot) = [-0.49, 3.18, -0.49, 0.06] \checkmark$

Verifying $V(s) = \sum_a \pi(a|s) \cdot Q(s, a)$:

Under a uniform random policy, $V(s)$ is simply the average of all Q-values:

$$\begin{aligned}
V(0, 2) &= \frac{1}{4}[Q(N) + Q(S) + Q(E) + Q(W)] \\
&= \frac{1}{4}[(-0.49) + 3.18 + (-0.49) + 0.06] \\
&= \frac{1}{4}[2.26] = 0.565 \approx 0.56 \checkmark
\end{aligned}$$

Key relationships demonstrated:

- i. $Q(s, a) = R + \gamma V(s')$ – the action-value equals immediate reward plus discounted value of the next state.
- ii. $V(s) = \sum_a \pi(a|s) \cdot Q(s, a)$ – the state-value is the policy-weighted average of action-values.
- iii. These two equations together give the full Bellman equation: $V(s) = \sum_a \pi(a|s)[R_a + \gamma V(s'_a)]$

Interpretation: The best action from (0, 2) is clearly South ($Q = 3.18$) since it leads toward the goal. Both boundary-hitting actions (N, E) are bad ($Q = -0.49$) because the agent pays a -1 penalty and stays in a low-value state. Under the uniform random policy, the agent picks the good action only 25% of the time, resulting in an overall $V(0, 2) = 0.56$ – much lower than the $Q = 3.18$ achievable by always going south.

Complete V and Q tables for the entire 3×3 grid:

State-Value Function $V(s)$:

	Col 0	Col 1	Col 2	
	-----+	-----+	-----+	
Row 0	-0.86	0.06	0.56	
	-----+	-----+	-----+	
Row 1	0.06	1.62	3.54	
	-----+	-----+	-----+	
Row 2	0.56	3.54	0.00	← Goal
	-----+	-----+	-----+	

Action-Value Function $Q(s, a)$ for all states:

State	$Q(s, N)$	$Q(s, S)$	$Q(s, E)$	$Q(s, W)$	$V(s) = \frac{1}{4} \sum Q$
(0, 0)	-1.77	0.06	0.06	-1.77	-0.86
(0, 1)	-0.94	1.46	0.51	-0.77	0.06
(0, 2)	-0.49	3.18	-0.49	0.06	0.56
(1, 0)	-0.77	0.51	1.46	-0.94	0.06
(1, 1)	0.06	3.18	3.18	0.06	1.62
(1, 2)	0.51	10.00	2.18	1.46	3.54
(2, 0)	0.06	-0.49	3.18	-0.49	0.56
(2, 1)	1.46	2.18	10.00	0.51	3.54
(2, 2)	—	—	—	—	0.00 (terminal)

Observations from the full tables:

- **Highest Q-values** ($Q = 10.00$) appear for actions that move directly into the goal: $Q((1, 2), S)$ and $Q((2, 1), E)$. These equal the goal reward $R = +10$ since $V(\text{terminal}) = 0$.
- **States adjacent to goal** — $(1, 2)$ and $(2, 1)$ — have the highest V-values (3.54) because they have a 25% chance of stepping into the goal each turn.
- **Symmetry** — the grid is symmetric along the diagonal from $(0, 0)$ to $(2, 2)$: $V(0, 1) = V(1, 0)$, $V(0, 2) = V(2, 0)$, $V(1, 2) = V(2, 1)$, etc.
- **Boundary penalties** — all negative Q-values correspond to actions that hit a wall ($R = -1$, stay in place). The worst is $Q((0, 0), N) = Q((0, 0), W) = -1.77$: hitting a wall from the corner of a low-value state.
- **Greedy policy** from the Q-table: always move toward $(2, 2)$ — South or East depending on position. This is the optimal policy for this grid.

12. **Question: Recycling Robot:** A robot is in state "high" (s). Action "search": $p(\text{high}, 3 \mid \text{high}, \text{search}) = 0.6$, $p(\text{low}, 3 \mid \text{high}, \text{search}) = 0.4$. Current values: $V(\text{high}) = 10$, $V(\text{low}) = 5$. Calculate $q(\text{high}, \text{search})$ with $\gamma = 0.8$.

Answer: $q(\text{high}, \text{search}) = 0.6[3 + 0.8(10)] + 0.4[3 + 0.8(5)] = 0.6[11] + 0.4[7] = 6.6 + 2.8 = 9.4$.

Explanation (Sutton & Barto, Ch 3, Section 3.6 - Example 3.3 Recycling Robot):

The action-value function is: $q_\pi(s, a) = \sum_{s', r} p(s', r \mid s, a)[r + \gamma V(s')]$

For the "search" action in state "high", there are two possible transitions:

- With probability 0.6: stay in "high", get reward 3 $\rightarrow$ contribution: $0.6[3 + 0.8 \times V(\text{high})] = 0.6[3 + 8] = 0.6 \times 11 = 6.6$
- With probability 0.4: transition to "low", get reward 3 $\rightarrow$ contribution: $0.4[3 + 0.8 \times V(\text{low})] = 0.4[3 + 4] = 0.4 \times 7 = 2.8$

Sum: $q(\text{high}, \text{search}) = 6.6 + 2.8 = 9.4$

The reward of 3 represents the expected search reward. The robot risks moving to "low" battery state (40% chance) where future value is lower.

13. **Question: Gambler's Problem:** If a gambler has capital $s = 50$, bets all of it ($a = 50$), and $p_{\text{win}} = 0.4$, calculate the expected value $V(50)$ given $V(0) = 0$ and $V(100) = 1$ with $\gamma = 1$.

Answer: $V(50) = 0.4[V(100)] + 0.6[V(0)] = 0.4(1) + 0.6(0) = 0.4$.

Explanation (Sutton & Barto, Ch 4, Section 4.4 - Example 4.3 Gambler's Problem):

The gambler bets all their capital ($a = 50$) on a coin flip:

- **Win** (prob = 0.4): capital goes from 50 to $50 + 50 = 100$ (goal reached). $V(100) = 1$ (terminal reward).
- **Lose** (prob = 0.6): capital goes from 50 to $50 - 50 = 0$ (gambler is ruined). $V(0) = 0$ (terminal, no reward).

Using the Bellman equation with $\gamma = 1$ (episodic, undiscounted):

$$V(s) = p_{\text{win}}[R + \gamma V(s + a)] + p_{\text{lose}}[R + \gamma V(s - a)]$$

Since rewards are 0 except at goal ($V(100) = 1$ encodes the reward):

$$V(50) = 0.4 \times V(100) + 0.6 \times V(0) = 0.4 \times 1 + 0.6 \times 0 = 0.4$$

This means going all-in from state 50 gives a 40% probability of reaching the goal — exactly p_{win} .

14. **Question: Small MDP Cycle:** State $A \rightarrow B$ with $R = 1$, $B \rightarrow A$ with $R = 1$. Calculate $V(A)$ and $V(B)$ for a policy that always moves, with $\gamma = 0.5$.

Answer: $V(A) = 1 + 0.5V(B)$; $V(B) = 1 + 0.5V(A)$.

$$V(A) = 1 + 0.5(1 + 0.5V(A)) = 1 + 0.5 + 0.25V(A) \Rightarrow 0.75V(A) = 1.5 \Rightarrow V(A) = 2. \text{ By symmetry, } V(B) = 2.$$

Explanation (Sutton & Barto, Ch 3, Section 3.5 - Bellman Equation as simultaneous equations):

This is a two-state continuing MDP with deterministic transitions:

- From A: take action $\rightarrow$ go to B, get $R = 1$
- From B: take action $\rightarrow$ go to A, get $R = 1$

The Bellman equation $V(s) = R + \gamma V(s')$ gives us two simultaneous equations:

- $V(A) = 1 + 0.5 \cdot V(B) \dots$ (i)
- $V(B) = 1 + 0.5 \cdot V(A) \dots$ (ii)

Solving by substitution – substitute (ii) into (i):

$$V(A) = 1 + 0.5(1 + 0.5 \cdot V(A))$$

$$V(A) = 1 + 0.5 + 0.25 \cdot V(A)$$

$$V(A) - 0.25 \cdot V(A) = 1.5$$

$$0.75 \cdot V(A) = 1.5$$

$$V(A) = 2$$

$$\text{Substituting back: } V(B) = 1 + 0.5 \times 2 = 2$$

Intuition: Both states are symmetric (each transitions to the other with the same reward), so they must have the same value. The value of 2 represents the discounted sum of all future rewards: $1 + 0.5 + 0.25 + \dots = \frac{1}{1-0.5} = 2$.

15. **Question: Bellman Optimality:** In a state S , two actions a_1, a_2 exist. a_1 : 50% chance of $R = 10, V(S') = 0$; 50% chance of $R = 0, V(S') = 10$. a_2 : 100% chance of $R = 4, V(S') = 2$. With $\gamma = 1$, find $V^*(S)$.

Answer: $a_1 : 0.5[10 + 0] + 0.5[0 + 10] = 5 + 5 = 10$.

$$a_2 : 1.0[4 + 2] = 6.$$

$$V^*(S) = \max(10, 6) = 10.$$

Explanation (Sutton & Barto, Ch 3, Section 3.6 - Bellman Optimality Equation):

The Bellman optimality equation is: $V^*(s) = \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma V^*(s')]$

We compute $q^*(s, a)$ for each action, then take the max.

Action a_1 (stochastic – two outcomes):

- 50% chance: $R = 10$, next state value $V(S') = 0 \rightarrow$ contribution: $0.5[10 + 1 \times 0] = 5$
- 50% chance: $R = 0$, next state value $V(S') = 10 \rightarrow$ contribution: $0.5[0 + 1 \times 10] = 5$
- Total: $q^*(s, a_1) = 5 + 5 = 10$

Action a_2 (deterministic – one outcome):

- 100% chance: $R = 4$, next state value $V(S') = 2 \rightarrow q^*(s, a_2) = 1.0[4 + 1 \times 2] = 6$

Optimal value: $V^*(S) = \max(10, 6) = 10 \rightarrow$ the optimal policy chooses a_1 .

Note: $\gamma = 1$ here, so there is no discounting. Even though a_1 is risky (stochastic), its expected value is higher than the safe action a_2 .

16. **Question: Effective Horizon:** If $\gamma = 0.9$, at what step k does the reward R_{t+k+1} contribute less than 10% of its original value to the return G_t ?

Answer: $\gamma^k < 0.1 \Rightarrow 0.9^k < 0.1$.

$$k \log(0.9) < \log(0.1) \Rightarrow k(-0.0457) < -1 \Rightarrow k > 1/0.0457 \approx 21.8. \text{ So } k = 22.$$

Explanation (Sutton & Barto, Ch 3, Section 3.3 - Effective Horizon):

In the return $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$, the reward k steps in the future is weighted by γ^k .

We want to find when this weight drops below 10% (i.e., the reward contributes less than 10% of its face value):

$$\gamma^k < 0.1$$

Taking logarithm of both sides:

$$k \cdot \log(0.9) < \log(0.1)$$

Since $\log(0.9) \approx -0.0457$ (negative), dividing by a negative flips the inequality:

$$k > \frac{\log(0.1)}{\log(0.9)} = \frac{-1}{-0.0457} \approx 21.85$$

So $k = 22$ is the first step where the discount factor makes the reward worth less than 10%.

Intuition: This defines the "effective horizon" – with $\gamma = 0.9$, the agent effectively looks ~22 steps ahead. Beyond that, future rewards are so heavily discounted they barely matter. This is why γ controls the agent's "farsightedness."

17. **Question: Probability Derivation:** $p(s', r | s, a)$ is given for $r \in \{0, 10\}$. If $\sum_{s'} p(s', 10 | s, a) = 0.2$ and $\sum_{s'} p(s', 0 | s, a) = 0.8$, calculate $r(s, a)$.

Answer: $r(s, a) = \sum_r r \sum_{s'} p(s', r | s, a) = 10(0.2) + 0(0.8) = 2$.

Explanation (Sutton & Barto, Ch 3, Section 3.2 - Eq. 3.5):

The expected reward for a state-action pair is defined as:

$$r(s, a) = E[R_{t+1} | S_t = s, A_t = a] = \sum_r r \sum_{s'} p(s', r | s, a)$$

This marginalizes out the next state s' to get the probability of each reward value:

- $\sum_{s'} p(s', r = 10 | s, a) = 0.2$ — across all possible next states, reward 10 occurs with total probability 0.2
- $\sum_{s'} p(s', r = 0 | s, a) = 0.8$ — reward 0 occurs with total probability 0.8

Therefore: $r(s, a) = 10 \times 0.2 + 0 \times 0.8 = 2 + 0 = 2$

Key distinction: The full dynamics $p(s', r | s, a)$ specifies probabilities over (next-state, reward) pairs. The expected reward $r(s, a)$ collapses this into a single number by summing over all s' and weighting each r by its marginal probability.

18. **Question: Policy Improvement:** Policy π gives $V_\pi(s) = 10$. A new action a' in state s has $q_\pi(s, a') = 12$. If we change $\pi(s)$ to a' , what is the lower bound for the new value $V_{\pi'}(s)$ according to the Policy Improvement Theorem?

Answer: $V_{\pi'}(s) \geq q_\pi(s, \pi'(s)) = 12$. The lower bound is 12.

Explanation (Sutton & Barto, Ch 4, Section 4.2 - Policy Improvement Theorem):

The Policy Improvement Theorem states: If for all s ,

$$q_\pi(s, \pi'(s)) \geq V_\pi(s)$$

then the new policy π' is at least as good as π : $V_{\pi'}(s) \geq V_\pi(s)$ for all s .

In this problem:

- Current policy π gives $V_\pi(s) = 10$
- A new action a' has $q_\pi(s, a') = 12$
- Since $q_\pi(s, a') = 12 > 10 = V_\pi(s)$, switching to a' in state s is an improvement

The theorem guarantees: $V_{\pi'}(s) \geq q_\pi(s, \pi'(s)) = 12$

Why it's a lower bound (not exact): The value under π' could be even higher than 12 because the improvement might cascade — changing the action in state s might lead to better states, whose improved values further increase $V_{\pi'}(s)$. The theorem gives a one-step-lookahead guarantee, but the actual improvement could be larger.

This is the foundation of policy iteration: greedily improve the policy at each state where $q_\pi(s, a) > V_\pi(s)$ for some action a .

19. **Question: Two-State Evaluation:** $S = \{1, 2\}$. $\pi(1) = a$, $\pi(2) = b$. $1 \xrightarrow{a} 2$ with $R = 10$. $2 \xrightarrow{b} 1$ with $R = 0$. $\gamma = 0.9$. Solve for $V(1)$ and $V(2)$.

Answer: $V(1) = 10 + 0.9V(2)$; $V(2) = 0 + 0.9V(1)$.

$$V(1) = 10 + 0.9(0.9V(1)) = 10 + 0.81V(1) \Rightarrow 0.19V(1) = 10 \Rightarrow V(1) \approx 52.63.$$

$$V(2) = 0.9(52.63) \approx 47.37.$$

Explanation (Sutton & Barto, Ch 4, Section 4.1 - Policy Evaluation as linear system):

This is a two-state continuing MDP with a fixed policy:

- State 1: action a deterministically goes to state 2 with reward $R = 10$
- State 2: action b deterministically goes to state 1 with reward $R = 0$

The Bellman equations form a system of linear equations:

- $V(1) = R_1 + \gamma V(2) = 10 + 0.9 \cdot V(2) \dots$ (i)
- $V(2) = R_2 + \gamma V(1) = 0 + 0.9 \cdot V(1) \dots$ (ii)

Solving by substitution — substitute (ii) into (i):

$$V(1) = 10 + 0.9 \times (0.9 \cdot V(1))$$

$$V(1) = 10 + 0.81 \cdot V(1)$$

$$V(1) - 0.81 \cdot V(1) = 10$$

$$0.19 \cdot V(1) = 10$$

$$V(1) = 10/0.19 \approx 52.63$$

$$\text{Back-substitute: } V(2) = 0.9 \times 52.63 \approx 47.37$$

Why the values are so large: The agent cycles forever ($1 \rightarrow 2 \rightarrow 1 \rightarrow 2 \rightarrow \dots$), collecting reward 10 every other step. With $\gamma = 0.9$ (close to 1), rewards far in the future still matter significantly, accumulating to a large total.

Compare to Q14: Same structure but here rewards are asymmetric (10 and 0 vs 1 and 1) and $\gamma = 0.9$ vs 0.5, leading to much larger values.

20. **Question: Max Stake:** In Gambler's problem with $s = 60$ and goal 100, what is the maximum possible stake a if the gambler cannot bet more than their current capital and cannot bet more than what is needed to reach the goal?

Answer: Stake a must be $\leq s$ and $\leq 100 - s$.

$$a \leq 60 \text{ and } a \leq 100 - 60 = 40. \text{ Max stake} = 40.$$

Explanation (Sutton & Barto, Ch 4, Section 4.4 - Example 4.3 Gambler's Problem):

In the Gambler's Problem, the action space (possible stakes) is constrained by two rules:

- i. **Cannot bet more than you have:** $a \leq s$ (you can't wager money you don't possess)
- ii. **Cannot bet more than needed to reach the goal:** $a \leq 100 - s$ (winning more than the goal is pointless/not allowed)

So the valid action space is: $a \in \{1, 2, \dots, \min(s, 100 - s)\}$

For $s = 60$:

- Constraint 1: $a \leq 60$
- Constraint 2: $a \leq 100 - 60 = 40$ (if the gambler bets 40 and wins, they reach exactly 100)

$$\text{Maximum stake} = \min(60, 40) = 40$$

Intuition: The gambler needs only 40 more to reach the goal of 100. Betting more than 40 would overshoot the goal if they win, so it's not permitted. This constraint shapes the optimal policy — near the goal, the action space shrinks.

21. **Question:** In ϵ -greedy action selection, for the case of two actions and $\epsilon = 0.5$, what is the probability that the greedy action is selected?

Answer: p (greedy action)

$$= p(\text{greedy action AND greedy selection}) + p(\text{greedy action AND random selection})$$

$$= p(\text{greedy action} \mid \text{greedy selection}) p(\text{greedy selection}) + p(\text{greedy action} \mid \text{random selection}) p(\text{random selection})$$

$$= p(\text{greedy action} \mid \text{greedy selection}) (1 - \epsilon) + p(\text{greedy action} \mid \text{random selection}) (\epsilon)$$

$$= p(\text{greedy action} \mid \text{greedy selection}) (0.5) + p(\text{greedy action} \mid \text{random selection}) (0.5)$$

$$= (1) (0.5) + (0.5) (0.5)$$

$$= 0.5 + 0.25$$

$$= 0.75$$

22. **Question: Policy Iteration vs Value Iteration:** Consider a 3-state MDP with states $\{A, B, \text{Terminal}\}$ and two actions (left, right) per state. The dynamics are:

- From A : **left** $\rightarrow$ stay at A , $R = -1$; **right** $\rightarrow$ go to B , $R = 0$
- From B : **left** $\rightarrow$ go to A , $R = 0$; **right** $\rightarrow$ go to Terminal, $R = +5$
- $\gamma = 0.9$

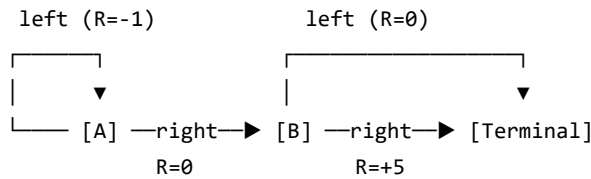
(a) Policy Iteration from $\pi(A) = \text{left}$, $\pi(B) = \text{left}$.

(b) Value Iteration from $V(A) = 0$, $V(B) = 0$.

(c) Compare the two methods.

Answer (Sutton & Barto, Ch 4, Sections 4.2-4.4):

MDP Diagram:



(a) POLICY ITERATION:

Iteration 1 — Policy Evaluation (solve Bellman equations exactly for $\pi(A) = \text{left}$, $\pi(B) = \text{left}$):

Under this policy, the transitions are:

- $A \xrightarrow{\text{left}} A$ with $R = -1$
- $B \xrightarrow{\text{left}} A$ with $R = 0$

Bellman equations:

- $V(A) = -1 + 0.9 \cdot V(A) \implies 0.1 \cdot V(A) = -1 \implies V(A) = -10$
- $V(B) = 0 + 0.9 \cdot V(A) = 0.9 \times (-10) = -9$

Iteration 1 — Policy Improvement (compute Q-values, pick argmax):

State	$Q(s, \text{left})$	$Q(s, \text{right})$	New $\pi(s)$
A	$-1 + 0.9(-10) = -10$	$0 + 0.9(-9) = -8.1$	right (improved!)
B	$0 + 0.9(-10) = -9$	$5 + 0.9(0) = 5$	right (improved!)

New policy: $\pi(A) = \text{right}$, $\pi(B) = \text{right}$

Iteration 2 — Policy Evaluation (solve for new policy):

- $V(B) = 5 + 0.9 \times 0 = 5$ (goes to terminal)
- $V(A) = 0 + 0.9 \times V(B) = 0 + 0.9 \times 5 = 4.5$

Iteration 2 — Policy Improvement:

State	$Q(s, \text{left})$	$Q(s, \text{right})$	New $\pi(s)$
A	$-1 + 0.9(4.5) = 3.05$	$0 + 0.9(5) = 4.5$	right (no change)
B	$0 + 0.9(4.5) = 4.05$	$5 + 0.9(0) = 5$	right (no change)

Policy is stable! Converged in 2 iterations.

Final: $V^*(A) = 4.5$, $V^*(B) = 5.0$, $\pi^*(A) = \text{right}$, $\pi^*(B) = \text{right}$

(b) VALUE ITERATION:

Starting with $V(A) = 0$, $V(B) = 0$:

Sweep	$Q(A, L)$	$Q(A, R)$	$V(A) = \max$	$Q(B, L)$	$Q(B, R)$	$V(B) = \max$	Δ
1	$-1 + 0.9(0) = -1$	$0 + 0.9(0) = 0$	0	$0 + 0.9(0) = 0$	$5 + 0 = 5$	5	5.0
2	$-1 + 0.9(0) = -1$	$0 + 0.9(5) = 4.5$	4.5	$0 + 0.9(0) = 0$	$5 + 0 = 5$	5	4.5
3	$-1 + 0.9(4.5) = 3.05$	$0 + 0.9(5) = 4.5$	4.5	$0 + 0.9(4.5) = 4.05$	$5 + 0 = 5$	5	0

Converged in 3 sweeps ($\Delta = 0$ at sweep 3).

Final: $V^*(A) = 4.5$, $V^*(B) = 5.0$

Extract policy: $\pi^*(s) = \arg \max_a Q(s, a) \rightarrow \pi^*(A) = \text{right}, \pi^*(B) = \text{right}$

(c) Comparison:

Aspect	Policy Iteration	Value Iteration
Iterations to converge	2	3 sweeps
Work per iteration	Expensive: solve system of linear equations (exact evaluation)	Cheap: one max operation per state
When policy found	After iteration 1 improvement (correct policy found early!)	After all sweeps converge (policy extracted at end)
Total computations	2 evaluations (solving n equations) + 2 improvements	3 sweeps $\times$ 2 states $\times$ 2 actions = 12 Q-value computations
Requires	Solving linear systems (or many iterative sweeps)	Only local max operations

Key insight: Policy Iteration found the optimal policy after just **1 improvement step** — even though the values were very wrong ($V(A) = -10$), the relative ordering of Q-values was already correct. Value Iteration took 3 sweeps but each sweep is computationally simpler.

(d) Scaling up — 5-state chain showing the real cost of Policy Iteration:

The 2-state example above used **exact** evaluation (solving linear equations directly). In practice, Policy Evaluation is done **iteratively** (repeated sweeps). This reveals the true cost difference.

Consider a 5-state chain $\{A, B, C, D, E, \text{Terminal}\}$ with the same action structure:

- **left:** stay in place, $R = -1$
- **right:** move one step toward Terminal, $R = 0$ (except $E \rightarrow \text{Terminal}$ gives $R = +10$)
- $\gamma = 0.9$

Optimal values: $V^* = [6.56, 7.29, 8.10, 9.00, 10.00]$

Policy Iteration (iterative evaluation, $\theta = 0.01$):

PI Iteration	Policy	Eval sweeps needed	Resulting V
1	all-left (initial)	45 sweeps	$[-9.91, -9.91, -9.91, -9.91, -9.91]$
2	all-right (improved)	6 sweeps	$[6.56, 7.29, 8.10, 9.00, 10.00]$

Policy stable after iteration 2. **Total: 51 sweeps.**

The first evaluation alone took 45 sweeps because the "all-left" policy creates a self-loop ($V(s) = -1 + 0.9V(s)$) that converges very slowly from 0 toward -10 .

Value Iteration ($\theta = 0.01$):

Sweep	$V(A)$	$V(B)$	$V(C)$	$V(D)$	$V(E)$	Δ
1	0	0	0	0	10	10
2	0	0	0	9	10	9
3	0	0	8.1	9	10	8.1
4	0	7.29	8.1	9	10	7.29
5	6.56	7.29	8.1	9	10	6.56
6	6.56	7.29	8.1	9	10	0 ✓

Total: 6 sweeps. Reward information propagates backward one state per sweep.

The dramatic comparison:

Method	Total sweeps	Why
Policy Iteration	51	45 sweeps wasted evaluating the terrible initial policy to convergence
Value Iteration	6	Each sweep both evaluates AND improves — no wasted work
Ratio	8.5× more work for PI	

Why this happens: Policy Iteration insists on fully evaluating the current policy before improving it. If the current policy is bad (e.g., self-loops), evaluation converges very slowly to a very negative value — all of which is "wasted" since

the policy will be discarded immediately after improvement. Value Iteration avoids this by taking the $\max$ at every sweep, effectively doing "truncated evaluation" (1 sweep) followed by immediate improvement.

When PI wins instead: If you use **exact** (matrix) evaluation instead of iterative, PI solves the 5-state system instantly and converges in just 2 policy changes. PI is faster when: (1) the state space is small enough for matrix inversion, or (2) the initial policy is already close to optimal.

23. **Question: Policy Evaluation Convergence:** Using the same MDP as Q22, perform iterative policy evaluation for $\pi(A) = \text{right}, \pi(B) = \text{right}$ starting from $V(A) = 0, V(B) = 0$. Show each sweep until convergence ($\theta = 0.01$).

Answer (Sutton & Barto, Ch 4, Section 4.1 - Iterative Policy Evaluation):

Under policy $\pi(A) = \text{right}, \pi(B) = \text{right}$, the update rules are:

- $V(A) \leftarrow R(A, \text{right}) + \gamma \cdot V(B) = 0 + 0.9 \cdot V(B)$
- $V(B) \leftarrow R(B, \text{right}) + \gamma \cdot V(\text{terminal}) = 5 + 0.9 \times 0 = 5$

Sweep	$V(A)$	$V(B)$	Δ
Init	0	0	—
1	$0 + 0.9(0) = 0$	$5 + 0 = 5$	5.0
2	$0 + 0.9(5) = 4.5$	$5 + 0 = 5$	4.5
3	$0 + 0.9(5) = 4.5$	$5 + 0 = 5$	0 ✓

Converged in 3 sweeps to $V(A) = 4.5, V(B) = 5.0$.

Comparison with exact solution:

Solving the Bellman equations directly (as done in Policy Iteration):

- $V(B) = 5$ (immediate, since B goes to terminal)
- $V(A) = 0.9 \times V(B) = 4.5$

The exact solution gives the same answer instantly by solving the linear system. Iterative evaluation took 3 sweeps because information propagates one step per sweep: sweep 1 learns $V(B) = 5$, sweep 2 propagates this to $V(A) = 4.5$, sweep 3 confirms no further change.

General principle: In a chain of length n , iterative policy evaluation needs at least n sweeps for information to propagate from the terminal state to the farthest state. Exact (matrix) solution has no such propagation delay but costs $O(|S|^3)$ for the matrix inverse.

24. **Question: Stochastic Gridworld — Value Iteration with Noisy Movement:** Consider the classic 3×4 gridworld with stochastic movement (80% intended, 10% each perpendicular), a wall at $(1, 1)$, terminal $+1$ at $(0, 3)$, terminal -1 at $(1, 3)$, living reward -0.04 , and $\gamma = 0.9$.

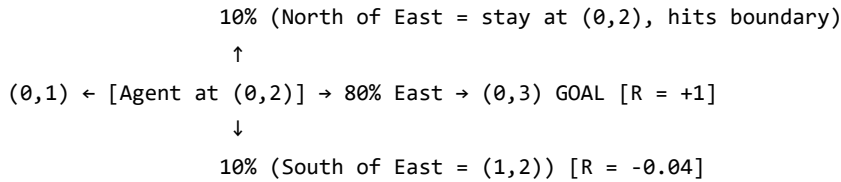
Answer (Sutton & Barto, Ch 4, Section 4.4 — Value Iteration; Russell & Norvig Ch 17):

The Grid:

	Col 0	Col 1	Col 2	Col 3
Row 0	(0,0)	(0,1)	(0,2)	+1 G
Row 1	(1,0)	■■■WALL■■■	(1,2)	-1 P
Row 2	(2,0)	(2,1)	(2,2)	(2,3)

Non-terminal states: $(0, 0), (0, 1), (0, 2), (1, 0), (1, 2), (2, 0), (2, 1), (2, 2), (2, 3)$ — 9 states.

(a) Stochastic Transitions — Example for $(0, 2)$ taking action East:



- **80%** $\rightarrow (0, 3)$: Intended direction. Enters terminal. Reward = +1.
- **10%** $\rightarrow$ **stays at** $(0, 2)$: Perpendicular (North). Hits top boundary, stays. Reward = -0.04 .
- **10%** $\rightarrow (1, 2)$: Perpendicular (South). Valid move. Reward = -0.04 .

Note: The reward for entering a terminal state is the terminal's value (+1 or -1). The reward for any other transition is the living reward -0.04 .

(b) Value Iteration — All Iterations:

Update rule: $V(s) \leftarrow \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma \cdot V(s')]$

Iteration 0 (initialization):

	Col 0	Col 1	Col 2	Col 3
Row 0	0.000	0.000	0.000	+1.000
Row 1	0.000	■■■WALL■■■	0.000	-1.000
Row 2	0.000	0.000	0.000	0.000

Iteration 1 ($\Delta = 0.792$):

	Col 0	Col 1	Col 2	Col 3
Row 0	-0.040	-0.040	+0.792	+1.000
Row 1	-0.040	■■■WALL■■■	-0.040	-1.000
Row 2	-0.040	-0.040	-0.040	-0.040

Only $(0, 2)$ gets a significant value: it can reach the +1 goal with 80% probability via East.

All other states can only reach non-terminal neighbors (all at $V = 0$), so they get -0.04 (just the living reward).

Iteration 2 ($\Delta = 0.563$):

	Col 0	Col 1	Col 2	Col 3
Row 0	-0.076	+0.523	+0.860	+1.000
Row 1	-0.076	WALL	+0.431	-1.000
Row 2	-0.076	-0.076	-0.076	-0.076

Value propagates from (0, 2) to its neighbors (0, 1) and (1, 2).

Iteration 3 ($\Delta = 0.399$):

	Col 0	Col 1	Col 2	Col 3
Row 0	+0.323	+0.673	+0.908	+1.000
Row 1	-0.108	WALL	+0.522	-1.000
Row 2	-0.108	-0.108	+0.256	-0.108

Iteration 4 ($\Delta = 0.281$):

	Col 0	Col 1	Col 2	Col 3
Row 0	+0.464	+0.735	+0.921	+1.000
Row 1	+0.173	WALL	+0.565	-1.000
Row 2	-0.138	+0.125	+0.316	+0.039

Iteration 5 ($\Delta = 0.152$):

	Col 0	Col 1	Col 2	Col 3
Row 0	+0.547	+0.755	+0.926	+1.000
Row 1	+0.325	WALL	+0.578	-1.000
Row 2	+0.083	+0.210	+0.381	+0.095

Iteration 6 ($\Delta = 0.087$):

	Col 0	Col 1	Col 2	Col 3
Row 0	+0.582	+0.762	+0.927	+1.000
Row 1	+0.412	WALL	+0.583	-1.000
Row 2	+0.221	+0.272	+0.403	+0.147

Iteration 7 ($\Delta = 0.041$):

	Col 0	Col 1	Col 2	Col 3
Row 0	+0.598	+0.765	+0.928	+1.000
Row 1	+0.453	WALL	+0.584	-1.000
Row 2	+0.301	+0.300	+0.417	+0.168

Iteration 8 ($\Delta = 0.020$):

	Col 0	Col 1	Col 2	Col 3
Row 0	+0.605	+0.766	+0.928	+1.000
Row 1	+0.473	WALL	+0.585	-1.000
Row 2	+0.341	+0.314	+0.423	+0.180

Iteration 9 ($\Delta = 0.009$):

	Col 0	Col 1	Col 2	Col 3
Row 0	+0.608	+0.766	+0.928	+1.000
Row 1	+0.481	WALL	+0.585	-1.000
Row 2	+0.359	+0.321	+0.425	+0.184

Iteration 10 ($\Delta = 0.004$):

	Col 0	Col 1	Col 2	Col 3
Row 0	+0.610	+0.766	+0.928	+1.000
Row 1	+0.485	WALL	+0.585	-1.000
Row 2	+0.368	+0.324	+0.427	+0.187

Iteration 11 ($\Delta = 0.002$):

	Col 0	Col 1	Col 2	Col 3
Row 0	+0.610	+0.766	+0.928	+1.000
Row 1	+0.486	WALL	+0.585	-1.000
Row 2	+0.371	+0.325	+0.427	+0.188

Iteration 12 ($\Delta = 0.001$):

	Col 0	Col 1	Col 2	Col 3
Row 0	+0.610	+0.766	+0.928	+1.000
Row 1	+0.487	WALL	+0.585	-1.000
Row 2	+0.373	+0.326	+0.427	+0.188

Iteration 13 — CONVERGED ($\Delta = 0.0007 < 0.001$):

	Col 0	Col 1	Col 2	Col 3
Row 0	+0.610	+0.766	+0.928	+1.000
Row 1	+0.487	WALL	+0.585	-1.000
Row 2	+0.373	+0.326	+0.427	+0.189

Converged in 13 iterations.

Key observations:

- Values propagate outward from the +1 goal like a "wave"
- State (0, 2) converges fastest (directly adjacent to goal)
- Bottom-left corner (2, 0) converges slowest (farthest from goal)
- State (2, 3) has low value despite being adjacent to the goal row — it's next to the -1 pit!
- The stochastic transitions slow convergence compared to deterministic (values "leak" sideways)

(c) Detailed Q-value computation for state (0, 2) at Iteration 2:

Using values from Iteration 1: $V(0, 2) = 0.792$, $V(0, 1) = -0.040$, $V(1, 2) = -0.040$.

$Q((0, 2), \text{East})$ – intended direction toward goal:

- 80% $\rightarrow (0, 3)$ terminal: $R = +1$, $V = 0 \rightarrow 0.8 \times (1 + 0.9 \times 0) = 0.800$
- 10% $\rightarrow$ stays (0, 2) (North hits boundary): $R = -0.04 \rightarrow 0.1 \times (-0.04 + 0.9 \times 0.792) = 0.067$
- 10% $\rightarrow (1, 2)$ (South): $R = -0.04 \rightarrow 0.1 \times (-0.04 + 0.9 \times (-0.04)) = -0.008$
- **Total:** $Q = 0.800 + 0.067 + (-0.008) = 0.860$

$Q((0, 2), \text{North})$ – hits boundary, mostly stays:

- 80% $\rightarrow$ stays (0, 2) (boundary): $R = -0.04 \rightarrow 0.8 \times (-0.04 + 0.9 \times 0.792) = 0.538$
- 10% $\rightarrow (0, 1)$ (West): $R = -0.04 \rightarrow 0.1 \times (-0.04 + 0.9 \times (-0.04)) = -0.008$
- 10% $\rightarrow (0, 3)$ terminal (East): $R = +1 \rightarrow 0.1 \times (1 + 0) = 0.100$
- **Total:** $Q = 0.538 + (-0.008) + 0.100 = 0.631$

$Q((0, 2), \text{South})$ – moves toward wall area:

- 80% $\rightarrow (1, 2)$: $R = -0.04 \rightarrow 0.8 \times (-0.04 + 0.9 \times (-0.04)) = -0.061$
- 10% $\rightarrow (0, 1)$ (West): $R = -0.04 \rightarrow 0.1 \times (-0.04 + 0.9 \times (-0.04)) = -0.008$
- 10% $\rightarrow (0, 3)$ terminal (East): $R = +1 \rightarrow 0.1 \times (1 + 0) = 0.100$
- **Total:** $Q = -0.061 + (-0.008) + 0.100 = 0.032$

$Q((0, 2), \text{West})$ – moves away from goal:

- 80% $\rightarrow (0, 1)$: $R = -0.04 \rightarrow 0.8 \times (-0.04 + 0.9 \times (-0.04)) = -0.061$
- 10% $\rightarrow$ stays (0, 2) (North, boundary): $R = -0.04 \rightarrow 0.1 \times (-0.04 + 0.9 \times 0.792) = 0.067$
- 10% $\rightarrow (1, 2)$ (South): $R = -0.04 \rightarrow 0.1 \times (-0.04 + 0.9 \times (-0.04)) = -0.008$
- **Total:** $Q = -0.061 + 0.067 + (-0.008) = -0.001$

Value Iteration picks the max:

$$V(0, 2) \leftarrow \max(0.860, 0.631, 0.032, -0.001) = 0.860 \quad (\text{action: East})$$

This matches Iteration 2's value for (0, 2).

(d) Optimal Policy (from converged values):

	Col 0	Col 1	Col 2	Col 3
Row 0	→ → → +1 G			
Row 1	↑ ■■■WALL■■■ ↑ -1 P			
Row 2	↑ → ↑ ←			

Policy interpretation:

- **Row 0:** All states go East $\rightarrow$ directly toward the +1 goal.
- **(1, 0):** Goes North to avoid getting pushed into the pit by stochastic movement.
- **(1, 2):** Goes North (toward (0, 2) which leads to goal). Does NOT go East – that would enter the –1 pit!
- **(2, 0), (2, 2):** Go North to reach Row 1/Row 0.

- (2, 1): Goes East (can't go North due to wall).
- (2, 3): Goes West ($\leftarrow$) to escape the neighborhood of the -1 pit. Going North would enter the pit!

Why (2, 3) goes Left: Even though the goal is "above" (2, 3), action North has 80% probability of entering the -1 pit at (1, 3). The agent prefers to retreat and take the longer safe path around.

Final Q-values for all states:

State	$Q(s, N)$	$Q(s, S)$	$Q(s, E)$	$Q(s, W)$	Best Action
(0, 0)	0.523	0.435	0.610	0.498	East
(0, 1)	0.650	0.650	0.766	0.537	East
(0, 2)	0.801	0.554	0.928	0.648	East
(1, 0)	0.487	0.317	0.399	0.399	North
(1, 2)	0.585	0.224	-0.686	0.503	North
(2, 0)	0.374	0.292	0.272	0.306	North
(2, 1)	0.267	0.267	0.326	0.288	East
(2, 2)	0.427	0.314	0.187	0.286	North
(2, 3)	-0.753	0.151	0.017	0.189	West

Note state (1, 2): $Q(\text{East}) = -0.686$ — going East has 80% chance of entering the -1 pit! The agent strongly avoids this action.

25. **Question: Discount Factor and Optimal Policy:** A 1D gridworld with 5 states: $a - b - c - d - e$. Actions: East, West, Exit (Exit only in terminals a and e). $R(a, \text{Exit}) = 10$, $R(e, \text{Exit}) = 1$. All moves give $R = 0$. Deterministic transitions. (a) Optimal policy for $\gamma = 1$? (b) Optimal policy for $\gamma = 0.1$? (c) For which γ are West and East equally good in state d ?

Answer (Sutton & Barto, Ch 3, Section 3.3 — The Role of Discounting):

Setup:

```
[a] --- [b] --- [c] --- [d] --- [e]
Exit=10                               Exit=1
```

The key insight: rewards are 0 for all movement. The agent only gets reward upon taking Exit. Discounting penalizes rewards that are farther in the future.

Value of going West (toward a) from any state:

- From b : takes 1 step $\rightarrow V = \gamma^1 \times 10 = 10\gamma$
- From c : takes 2 steps $\rightarrow V = \gamma^2 \times 10 = 10\gamma^2$
- From d : takes 3 steps $\rightarrow V = \gamma^3 \times 10 = 10\gamma^3$

Value of going East (toward e) from any state:

- From d : takes 1 step $\rightarrow V = \gamma^1 \times 1 = \gamma$
- From c : takes 2 steps $\rightarrow V = \gamma^2 \times 1 = \gamma^2$
- From b : takes 3 steps $\rightarrow V = \gamma^3 \times 1 = \gamma^3$

(a) $\gamma = 1$ — No discounting:

With no discounting, distance doesn't matter — only the reward magnitude:

- From b : West gives 10, East gives 1 $\rightarrow$ **West**
- From c : West gives 10, East gives 1 $\rightarrow$ **West**
- From d : West gives 10, East gives 1 $\rightarrow$ **West**
- a : **Exit** (collect reward 10)
- e : **Exit** (only option for a terminal)

Optimal policy: Everyone goes West toward the big reward.

$[a:\text{Exit}] \leftarrow [b] \leftarrow [c] \leftarrow [d] \leftarrow [e:\text{Exit}]$

(b) $\gamma = 0.1$ — Heavy discounting:

With $\gamma = 0.1$, future rewards decay extremely fast ($\gamma^2 = 0.01$, $\gamma^3 = 0.001$).

From d :

- West: $10 \times (0.1)^3 = 10 \times 0.001 = 0.01$
- East: $1 \times (0.1)^1 = 0.1$
- East wins! ($0.1 > 0.01$) $\rightarrow$ **East**

From c :

- West: $10 \times (0.1)^2 = 10 \times 0.01 = 0.1$
- East: $1 \times (0.1)^2 = 0.01$
- West wins! ($0.1 > 0.01$) $\rightarrow$ **West**

From b :

- West: $10 \times (0.1)^1 = 1.0$
- East: $1 \times (0.1)^3 = 0.001$
- West wins! ($1.0 > 0.001$) $\rightarrow$ **West**

Optimal policy: Nearby reward dominates; d goes East because e is closer.

$[a:\text{Exit}] \leftarrow [b] \leftarrow [c] \quad [d] \rightarrow [e:\text{Exit}]$

(c) Find γ where West = East in state d :

Set the values equal:

$$V_{\text{West}}(d) = V_{\text{East}}(d)$$

$$\gamma^3 \times 10 = \gamma^1 \times 1$$

Divide both sides by γ (valid since $\gamma > 0$):

$$10\gamma^2 = 1$$

$$\gamma^2 = \frac{1}{10} = 0.1$$

$$\gamma = \sqrt{0.1} = \frac{1}{\sqrt{10}} \approx 0.316$$

Verification:

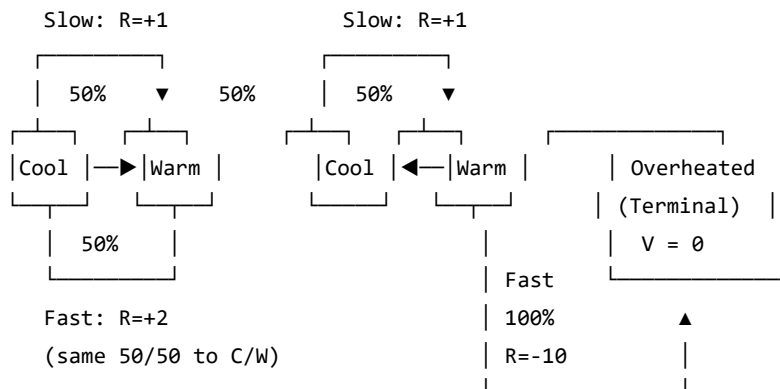
- West from d : $10 \times (0.316)^3 = 10 \times 0.0316 = 0.316$
- East from d : $1 \times (0.316)^1 = 0.316 \checkmark$

Interpretation: For $\gamma > 0.316$, the agent in d goes West (the big reward is worth the wait). For $\gamma < 0.316$, it goes East (the small reward nearby is better than a distant big one). This illustrates how γ controls the agent's "planning horizon" — low γ makes the agent myopic/greedy for immediate reward.

26. **Question: Race Car MDP — Policy Iteration vs Value Iteration:** A race car has states {Cool, Warm, Overheated (terminal)}. Actions: Slow ($R = +1$ to non-terminal), Fast ($R = +2$ to non-terminal, $R = -10$ to Overheated). Transitions: (Cool,Slow)→C/W 50/50; (Cool,Fast)→C/W 50/50; (Warm,Slow)→C/W 50/50; (Warm,Fast)→OH 100%. $\gamma = 0.9$. Solve with (a) Policy Iteration, (b) Value Iteration, (c) compare.

Answer (Sutton & Barto, Ch 4, Sections 4.2–4.4 — Policy Iteration & Value Iteration):

The MDP Diagram:



Key Q-value formulas:

$$Q(C, \text{Slow}) = 0.5(1 + 0.9 \cdot V(C)) + 0.5(1 + 0.9 \cdot V(W))$$

$$Q(C, \text{Fast}) = 0.5(2 + 0.9 \cdot V(C)) + 0.5(2 + 0.9 \cdot V(W))$$

$$Q(W, \text{Slow}) = 0.5(1 + 0.9 \cdot V(C)) + 0.5(1 + 0.9 \cdot V(W))$$

$$Q(W, \text{Fast}) = 1.0(-10 + 0.9 \cdot 0) = -10$$

Note: $Q(C, \text{Slow}) = Q(W, \text{Slow})$ always (same transitions and rewards). And $Q(C, \text{Fast}) = Q(C, \text{Slow}) + 1$ always (Fast just adds +1 more per transition).

(a) Policy Iteration:**Iteration 1 — Policy:** $\pi(C) = \text{Slow}, \pi(W) = \text{Slow}$ (**Cautious**)**Step 1: Policy Evaluation** (solve Bellman equations exactly):

Under this policy, both states always take Slow:

$$V(C) = 0.5(1 + 0.9 \cdot V(C)) + 0.5(1 + 0.9 \cdot V(W))$$

$$V(W) = 0.5(1 + 0.9 \cdot V(C)) + 0.5(1 + 0.9 \cdot V(W))$$

Both equations are **identical!** So $V(C) = V(W) = x$:

$$x = 0.5(1 + 0.9x) + 0.5(1 + 0.9x) = 1 + 0.9x$$

$$0.1x = 1 \implies \boxed{V(C) = V(W) = 10}$$

Step 2: Policy Improvement (compute Q-values, pick arg max):

(s, a)	Formula	Value
(C, Slow)	$0.5(1 + 9) + 0.5(1 + 9)$	10.0
(C, Fast)	$0.5(2 + 9) + 0.5(2 + 9)$	11.0
(W, Slow)	$0.5(1 + 9) + 0.5(1 + 9)$	10.0
(W, Fast)	$1.0(-10 + 0)$	-10.0

New policy: $\pi'(C) = \text{Fast}$ (11 > 10), $\pi'(W) = \text{Slow}$ (10 > -10).**Policy changed** from {Slow, Slow} $\rightarrow$ {Fast, Slow}. Continue.**Iteration 2 — Policy:** $\pi(C) = \text{Fast}, \pi(W) = \text{Slow}$ **Step 1: Policy Evaluation:**

$$V(C) = 0.5(2 + 0.9 \cdot V(C)) + 0.5(2 + 0.9 \cdot V(W))$$

$$V(W) = 0.5(1 + 0.9 \cdot V(C)) + 0.5(1 + 0.9 \cdot V(W))$$

Simplify:

- Eq 1: $V(C) = 2 + 0.45V(C) + 0.45V(W) \implies 0.55V(C) - 0.45V(W) = 2 \dots$ (i)
- Eq 2: $V(W) = 1 + 0.45V(C) + 0.45V(W) \implies -0.45V(C) + 0.55V(W) = 1 \dots$ (ii)

From (ii): $V(W) = \frac{1+0.45V(C)}{0.55}$

Substitute into (i):

$$0.55V(C) = 2 + 0.45 \cdot \frac{1 + 0.45V(C)}{0.55}$$

$$0.55V(C) = 2 + \frac{0.45 + 0.2025V(C)}{0.55}$$

$$0.3025V(C) = 1.1 + 0.45 + 0.2025V(C)$$

$$0.1V(C) = 1.55$$

$$V(C) = 15.5, \quad V(W) = 14.5$$

Step 2: Policy Improvement:

(s, a)	Formula	Value
(C, Slow)	$0.5(1 + 0.9 \times 15.5) + 0.5(1 + 0.9 \times 14.5)$	14.5
(C, Fast)	$0.5(2 + 0.9 \times 15.5) + 0.5(2 + 0.9 \times 14.5)$	15.5
(W, Slow)	$0.5(1 + 0.9 \times 15.5) + 0.5(1 + 0.9 \times 14.5)$	14.5
(W, Fast)	-10	-10

New policy: $\pi'(C) = \text{Fast}$, $\pi'(W) = \text{Slow}$ – **UNCHANGED. Converged!**

Policy Iteration: 2 iterations (one policy change).

(b) Value Iteration:

Starting from $V(C) = 0$, $V(W) = 0$. Update: $V(s) \leftarrow \max_a Q(s, a)$.

Iter	$Q(C, S)$	$Q(C, F)$	$V(C)$	$Q(W, S)$	$Q(W, F)$	$V(W)$	π^*	Δ
0	—	—	0	—	—	0	—	—
1	1.00	2.00	2.00	1.00	-10	1.00	F, S	2.00
2	2.35	3.35	3.35	2.35	-10	2.35	F, S	1.35
3	3.57	4.57	4.57	3.57	-10	3.57	F, S	1.22
4	4.66	5.66	5.66	4.66	-10	4.66	F, S	1.09
5	5.64	6.64	6.64	5.64	-10	5.64	F, S	0.98

Iter	$Q(C, S)$	$Q(C, F)$	$V(C)$	$Q(W, S)$	$Q(W, F)$	$V(W)$	π^*	Δ
10	9.27	10.27	10.27	9.27	-10	9.27	F, S	0.58
15	11.41	12.41	12.41	11.41	-10	11.41	F, S	0.34
20	12.68	13.68	13.68	12.68	-10	12.68	F, S	0.20
25	13.42	14.42	14.42	13.42	-10	13.42	F, S	0.12
30	13.86	14.86	14.86	13.86	-10	13.86	F, S	0.07
35	14.12	15.12	15.12	14.12	-10	14.12	F, S	0.04
40	14.28	15.28	15.28	14.28	-10	14.28	F, S	0.02
45	14.37	15.37	15.37	14.37	-10	14.37	F, S	0.01
49	14.41	15.41	15.41	14.41	-10	14.41	F, S	< 0.01 ✓

Value Iteration: 49 iterations to converge ($\theta = 0.01$).

Detailed computation for Iteration 1:

- $Q(C, \text{Slow}) = 0.5(1 + 0.9 \times 0) + 0.5(1 + 0.9 \times 0) = 0.5 + 0.5 = 1.0$
- $Q(C, \text{Fast}) = 0.5(2 + 0.9 \times 0) + 0.5(2 + 0.9 \times 0) = 1.0 + 1.0 = 2.0$
- $V(C) = \max(1.0, 2.0) = 2.0$ (pick Fast)
- $Q(W, \text{Slow}) = 0.5(1 + 0.9 \times 0) + 0.5(1 + 0.9 \times 0) = 1.0$
- $Q(W, \text{Fast}) = -10$
- $V(W) = \max(1.0, -10) = 1.0$ (pick Slow)

Detailed computation for Iteration 2:

- $Q(C, \text{Slow}) = 0.5(1 + 0.9 \times 2) + 0.5(1 + 0.9 \times 1) = 0.5(2.8) + 0.5(1.9) = 1.4 + 0.95 = 2.35$
- $Q(C, \text{Fast}) = 0.5(2 + 0.9 \times 2) + 0.5(2 + 0.9 \times 1) = 0.5(3.8) + 0.5(2.9) = 1.9 + 1.45 = 3.35$
- $V(C) = \max(2.35, 3.35) = 3.35$

(c) Comparison:

Method	Iterations to converge	Why
Policy Iteration	2	Exact evaluation solves the linear system in one shot; only 1 policy change needed
Value Iteration	49	Iteratively approaches V^* ; the self-loop ($C \rightarrow C$ with 50%) creates slow convergence
Ratio	24.5× more for VI	

Why Value Iteration is slow here:

The transition structure creates a "cycle": Cool $\rightarrow$ Cool with 50% probability. The Bellman update becomes approximately:

$$V(C) \approx 2 + 0.9 \times 0.5 \times V(C) + \dots = 2 + 0.45V(C) + \dots$$

The effective contraction rate is $\gamma \times P(\text{self-loop}) = 0.9 \times 0.5 = 0.45\dots$ but since both states feed each other, the actual convergence rate is governed by $\gamma = 0.9$ (the spectral radius). With $\gamma = 0.9$, each sweep reduces the error by at most a factor of 0.9, requiring $\approx \frac{\log(0.01)}{\log(0.9)} \approx 44$ sweeps to reduce error by 100 $\times$.

Why Policy Iteration is fast here:

- Exact evaluation solves the 2×2 linear system in one step (no iterative convergence needed)
- The initial cautious policy is only 1 action away from optimal (just need to switch Cool from Slow to Fast)
- After 1 improvement step, the optimal policy {Fast, Slow} is found

When each method wins:

- **PI wins** when: state space is small (matrix solve is cheap), or the initial policy is close to optimal
- **VI wins** when: state space is large (matrix solve is $O(|S|^3)$), or you only need an approximate answer quickly

The optimal policy (intuition):

- **Cool $\rightarrow$ Fast:** The car is cool, so go fast (+2 reward). Even if it warms up, there's no risk of overheating yet.
- **Warm $\rightarrow$ Slow:** The car is warm — going fast guarantees overheating ($R = -10$). Play it safe with Slow ($R = +1$) and hope to cool down (50% chance).

This makes physical sense: push hard when the engine is cool, back off when it's running hot.